

Git Engineering Handbook for DevOps

Architectural models, repository initialization, stage manipulation, and commit discipline.

1. Git Four-Tier Architecture & Data Transport Flow

INTERNAL STATE MACHINE

Git operates as a directed acyclic graph (DAG) of content-addressable objects (blobs, trees, commits, and annotated tags).

Data flows between four core environmental tiers:

Working Tree Live physical files on disk. Tracks modified and untracked entries. <code>git checkout / restore</code>	Staging (Index) Binary cache (<code>.git/index</code>) snapshotting upcoming commit state. <code>git add → index</code>	Local Repo (HEAD) Immutable commit graph recorded locally in <code>.git/objects</code> . <code>git commit → HEAD</code>	Remote (Origin) Upstream shared repository host (GitHub, GitLab, AWS CodeCommit). <code>git push / fetch</code>
--	--	---	--

2. Repository Setup, Config & CI/CD Environment Variables

ENVIRONMENT SETUP

Command	Syntax & Practical Execution	DevOps Scope & Real-World Example	Flags
<code>git config</code>	<code>git config --global user.name "Bot"</code>	Configures user identity, signing keys, and toolchain defaults across global or repo scope. Ex: <code>git config --global user.email "ci-runner@company.internal"</code>	<code>--global</code> , <code>--local</code> , <code>-l</code>
<code>git init</code>	<code>git init --initial-branch=main</code>	Initializes a new Git workspace, creating the internal <code>.git/</code> tracking tree. Ex: <code>git init --bare /srv/git/infra-deploy.git</code>	<code>--bare</code> , <code>-b</code>
<code>git clone</code>	<code>git clone --depth 1 <repo-url></code>	Clones upstream repo. <code>--depth 1</code> (shallow clone) truncates history for faster CI builds. Ex: <code>git clone --depth 1 -b release/v2.1 git@github.com:org/k8s.git</code>	<code>--depth</code> , <code>--branch</code>
<code>.gitignore</code>	<code>cat <<EOF > .gitignore</code>	Excludes build artifacts, secrets (<code>.env</code>), Terraform states, and dependencies. Ex: <code>echo -e ".terraform/\n*.tfstate\n.env" >> .gitignore</code>	Pattern matching

3. Daily Staging Lifecycle, Status Audits & Commits

WORKING STATE

Command	Syntax & Practical Execution	DevOps Scope & Real-World Example	Flags
<code>git status</code>	<code>git status -s -b</code>	Displays state of working tree and index. Short format (<code>-s</code>) parses quickly in automation. Ex: <code>git status -s -b # Shows: ## main...origin/main [ahead 1] M app.py</code>	<code>-s</code> , <code>-b</code> , <code>-uall</code>
<code>git add</code>	<code>git add -p [file]</code>	Stages changes into the index. Interactive patch mode (<code>-p</code>) stages atomic hunks selectively.	<code>-p</code> , <code>-A</code> , <code>-u</code> , <code>.</code>

4. Branch Management & Context Switching

ISOLATION & WORKFLOWS

Branches are lightweight movable pointers to commit hashes. Modern Git provides dedicated commands (`git switch` and `git restore`) to replace overloaded historical checkout operations:

Command	Syntax & Practical Execution	DevOps Scope & Real-World Example	Flags
<code>git branch</code>	<code>git branch -vv -a</code>	Lists local and remote branches with tracking status and ahead/behind counts. Ex: <code>git branch --merged main # Lists branches safe to prune</code>	<code>-a, -vv, -d, -D</code>
<code>git switch</code>	<code>git switch -c feat/k8s-ingress</code>	Switches active working context. Flag <code>-c</code> creates and checks out the branch atomically. Ex: <code>git switch -c hotfix/patch-log4j main</code>	<code>-c, -C, --detach</code>
<code>git branch -d</code>	<code>git branch -d [branch-name]</code>	Safely deletes local branch after upstream merge verification. <code>-D</code> forces deletion. Ex: <code>git branch -D obsolete/spike-experiment</code>	<code>-d, -D, -m</code>

5. Integrating Code: Fast-Forward, True Merge & Rebase

INTEGRATION PATTERNS

Merge (<code>git merge --no-ff</code>) Creates a non-destructive merge commit with two parents. Preserves full chronological history and exact feature branch boundaries. Ex: <code>git merge --no-ff feature/redis-cluster -m "merge: add redis"</code>	Rebase (<code>git rebase main</code>) Re-applies local commits on top of upstream tip. Creates linear history without extra merge nodes. Requires conflict resolution per commit. Ex: <code>git rebase origin/main # Aligns feature tip with latest trunk</code>
--	--

Command	Syntax & Practical Execution	DevOps Scope & Real-World Example	Flags
<code>git merge</code>	<code>git merge --squash feature/auth</code>	Combines an entire multi-commit branch history into one staged diff before merging to production. Ex: <code>git merge --squash feat/auth && git commit -m "feat: oauth2 flow"</code>	<code>--squash, --no-ff</code>
<code>git rebase -i</code>	<code>git rebase -i HEAD~4</code>	Interactive rebase: squash intermediate commits, rewrite commit messages, drop obsolete changes. Ex: <code>git rebase -i origin/main # Squashes dirty WIP commits before PR</code>	<code>-i, --onto, --abort</code>

6. Remote Synchronization & Upstream Collaboration

CI/CD INTEGRATION

Command	Syntax & Practical Execution	DevOps Scope & Real-World Example	Flags
<code>git remote</code>	<code>git remote -v</code>	Lists tracked upstream repo aliases alongside fetch/push URLs. Adds new cloud remotes. Ex: <code>git remote add upstream git@gitlab.internal:devops/shared.git</code>	<code>-v, add, prune</code>
<code>git fetch</code>	<code>git fetch --prune origin</code>	Downloads remote objects without altering working tree. Cleans obsolete remote branch tags.	<code>--prune, --all</code>

7. History Inspection, Log Graphs & Blame Forensics

AUDITING

Tracking production regressions, release tags, and identifying authors of specific lines of infrastructure code:

Command	Syntax & Practical Execution	DevOps Scope & Real-World Example	Flags
git log	<code>git log --oneline --graph --all -15</code>	Visualizes branches, merges, and tags in ASCII graph format across all local branches. Ex: <code>git log -n 5 --pretty=format:"%h - %an: %s (%cr)"</code>	<code>--oneline, -graph, -n</code>
git log -S	<code>git log -S "DB_PASSWORD" -p</code>	Pickaxe search: scans whole repository history for commits that added or deleted a text snippet. Ex: <code>git log -S "image: redis:6.0" -p charts/values.yaml</code>	<code>-S, -G, -p</code>
git blame	<code>git blame -L 40,65 Dockerfile</code>	Annotates specific line ranges with commit hash, author, and date of last change. Ex: <code>git blame -L 12,20 -w main.tf # Ignores whitespace diffs</code>	<code>-L, -e, -w</code>
git show	<code>git show v1.4.0:k8s/deploy.yaml</code>	Displays commit diffs or extracts file content directly from specific tags or past commits. Ex: <code>git show HEAD~2:terraform/vpc.tf > vpc_old.tf</code>	<code>--stat, [tag]:[file]</code>

8. Disaster Recovery, Undoing Changes & Reflog

STATE RESTORATION

Command	Syntax & Practical Execution	DevOps Scope & Real-World Example	Flags
git restore	<code>git restore --staged <file></code>	Unstages changes without losing edits. Omitting <code>--staged</code> reverts working tree to HEAD. Ex: <code>git restore --source=HEAD~1 k8s/service.yaml # Reverts single file</code>	<code>--staged, --source</code>
git reset	<code>git reset --soft HEAD~1</code>	Rolls back commits. <code>--soft</code> keeps changes staged; <code>--hard</code> permanently discards uncommitted work. Ex: <code>git reset --hard origin/main # Forces local branch to match remote</code>	<code>--soft, --mixed, --hard</code>
git revert	<code>git revert <commit-sha></code>	Creates an inverse commit to safely undo breaking changes without rewriting public history. Ex: <code>git revert -m 1 <merge-commit-sha> # Safely reverts a bad merge</code>	<code>--no-commit, -m</code>
git reflog	<code>git reflog show HEAD</code>	Audits all local HEAD movements. Recovers commits discarded by accidental <code>reset --hard</code> . Ex: <code>git reset --hard HEAD@{3} # Recovers state before failed rebase</code>	<code>show, expire</code>

9. Stashing & Cherry-Picking

HOTFIXES & SHELIVING

Stashing (	<code>git stash push -m "wip"</code>	)	Cherry-Picking (	<code>git cherry-pick <SHA></code>	)
------------	--------------------------------------	---	------------------	--	---